



SPRINT 2 • CAPÍTULO 2 • ANÁLISE DOS RESULTADOS

Do Texto Bruto à Entrada do Modelo

Implementação e experimentação
do pipeline de pré-processamento
de dados de um LLM

TOKENIZAÇÃO • TOKEN IDS • EMBEDDINGS • POSITIONAL EMBEDDINGS • DATALOADER

Matheus Dapper
Vitória Aparecida Vendausen

2026

Sprint 2 — 27 de agosto

Sumário

Sobre este documento	1
Questão 1 — Por que um LLM não pode trabalhar diretamente com o texto bruto?	1
Questão 2 — Qual é a função do vocabulário?	1
Questão 3 — Qual é a diferença entre um token e um Token ID?	1
Questão 4 — Por que os Token IDs não são utilizados diretamente como representação semântica?	2
Questão 5 — Qual é a função dos embeddings?	2
Questão 6 — Por que é necessário representar a posição dos tokens?	2
Questão 7 — Qual é a relação entre tamanho do contexto e quantidade de amostras de treinamento?	2
Questão 8 — Qual é o impacto da dimensão do embedding sobre as estruturas utilizadas pelo modelo?	3
Questão 9 — Qual é a função do DataLoader no pipeline?	3
Questão 10 — Quais informações produzidas nesta Sprint serão utilizadas pelo mecanismo de atenção da Sprint seguinte?	4

Sobre este documento

Este documento apresenta a análise técnica dos resultados obtidos na Sprint 2 do Projeto LLM, referente ao Capítulo 2 — *Working with Text Data* — do livro *Build a Large Language Model (From Scratch)* (Raschka, 2025). A implementação completa encontra-se em `src/` (tokenização, vocabulário, dataset, embeddings) e os experimentos, em `experiments/run_experiments.py`, cujos resultados brutos estão em `experiments/resultados_experimentos.md`. Os valores numéricos citados abaixo foram extraídos diretamente dessa execução, sobre o corpus de referência do capítulo (*The Verdict*, de Edith Wharton — 20 479 caracteres, 5 145 tokens BPE).

Questão 1. Por que um LLM não pode trabalhar diretamente com o texto bruto?

Uma rede neural, em sua essência, é uma composição de operações matemáticas — somas, produtos matriciais, funções não lineares — que operam sobre tensores de números reais. Uma *string* de caracteres não é uma estrutura numérica: não há uma operação de “multiplicar” ou “somar” definida entre as palavras “gato” e “cachorro”. É necessário, portanto, um pipeline de conversão que traduza texto em estrutura numérica antes de qualquer camada da rede poder processá-lo.

Esse pipeline é exatamente o assunto desta Sprint: *Tokenização* → *Token IDs* → *Embeddings*. Cada etapa reduz uma forma de representação a outra mais tratável computacionalmente, até chegar a um tensor contínuo — o *input embedding* — que de fato serve de entrada aos blocos do Transformer.

Questão 2. Qual é a função do vocabulário?

O vocabulário é o dicionário finito que define *todos* os tokens que o modelo é capaz de reconhecer, associando cada um a um Token ID único. Ele cumpre um papel duplo:

- **Na entrada**, é o que permite converter qualquer texto em uma sequência de inteiros — sem vocabulário, não há como decidir que número representa cada token;
- **Na saída**, define a dimensão do vetor de *logits* produzido pela última camada do modelo: prever o “próximo token” significa, na prática, prever uma distribuição de probabilidade sobre todas as entradas do vocabulário.

No experimento realizado, o vocabulário construído a partir do corpus *The Verdict* (tokens únicos, ordenados alfabeticamente) possui 1 130 entradas — 1 132 somando os tokens especiais `<|unk|>` e `<|endof-text|>`. Já o vocabulário BPE do GPT-2, fixo e independente do corpus, possui 50 257 entradas. Essa diferença de mais de 40x ilustra a diferença entre um vocabulário *ad hoc* (dependente do corpus de treino) e um vocabulário de subpalavras genérico, construído uma única vez sobre um corpus massivo e reutilizável para qualquer texto em inglês.

Questão 3. Qual é a diferença entre um token e um Token ID?

O **token** é a unidade textual — uma *string* como `"the"`, `"sun"` ou `" , "` — produzida pela tokenização. O **Token ID** é o número inteiro que representa esse token dentro de um vocabulário específico: é o índice da linha correspondente na tabela de vocabulário.

A relação não é intrínseca ao token, mas depende do vocabulário usado: o mesmo token `"the"` pode ter o ID 262 no vocabulário BPE do GPT-2 e um ID completamente diferente no vocabulário construído a partir de *The Verdict*. O token é a unidade *linguística*; o Token ID é a unidade *computacional* — só faz sentido em relação ao vocabulário que o define.

Questão 4. Por que os Token IDs não são utilizados diretamente como representação semântica?

Um Token ID é apenas um índice arbitrário de posição em uma tabela — geralmente atribuído por ordem alfabética ou por frequência de mesclagem, no caso do BPE. Não há nenhuma relação matemática pretendida entre a magnitude ou proximidade de dois IDs e a proximidade semântica dos tokens que eles representam.

Por exemplo, no vocabulário construído neste projeto, os tokens "!" (ID 0) e "%" (ID muito próximo, adjacente na ordenação alfabética de símbolos) estão numericamente vizinhos apenas por coincidência de ordenação — não por qualquer relação de sentido. Se a rede usasse o ID diretamente como entrada numérica de uma camada linear, ela seria forçada a interpretar “304 é mais parecido com 305 do que com 9000” como um fato semântico, o que é falso e arbitrário. É exatamente esse problema que a camada de *Embedding* resolve: substitui o índice por um vetor treinável, cujas distâncias no espaço vetorial *podem* ser aprendidas para refletir relações reais de uso e significado.

Questão 5. Qual é a função dos embeddings?

O embedding troca a representação de índice inteiro por um vetor denso de números reais, de dimensão fixa (`output_dim`). Esse vetor é um parâmetro *treinável* da rede — inicializado aleatoriamente e ajustado durante o treinamento — e é sobre ele, e não mais sobre o Token ID bruto, que as camadas seguintes (atenção, *feed-forward*) efetivamente operam.

No experimento com `output_dim=256`, cada Token ID (um único inteiro) passa a ser representado por um vetor de 256 valores reais. A tabela de embeddings completa, para o vocabulário do GPT-2, tem $50\,257 \times 256 = 12\,865\,792$ parâmetros treináveis — ou seja, mais de 12,8 milhões de números só para mapear tokens em vetores, antes mesmo de qualquer bloco de atenção existir. Esse número escala linearmente com `output_dim`: com 16 dimensões, a mesma tabela cai para 804 112 parâmetros; com 768 dimensões (padrão do GPT-2 pequeno), sobe para 38 597 376.

Questão 6. Por que é necessário representar a posição dos tokens?

A camada de embedding é uma tabela de consulta indexada por Token ID: o mesmo token gera sempre o mesmo vetor, *independentemente da posição em que aparece na sequência*. Sem uma correção para isso, o modelo enxergaria a entrada como um *conjunto* de tokens (um *bag of tokens*), não como uma sequência ordenada — e “o cachorro mordeu o homem” se tornaria indistinguível de “o homem mordeu o cachorro”.

O *positional embedding* resolve isso somando, a cada token embedding, um segundo vetor que depende apenas da posição (0, 1, 2, ...) dentro do contexto, obtido de uma segunda tabela treinável de tamanho (`context_length`, `output_dim`). O experimento em `src/embeddings.py` confirma isso diretamente: o mesmo Token ID (40) somado ao positional embedding da posição 0 produz um vetor diferente do mesmo Token ID somado ao positional embedding da posição 2 (`vetor_pos0 != vetor_pos2`). É essa soma — *Input embedding* = token embedding + positional embedding — que efetivamente entra nos blocos do Transformer.

Questão 7. Qual é a relação entre tamanho do contexto e quantidade de amostras de treinamento?

Para um corpus de tamanho fixo, quanto maior o tamanho do contexto (`max_length`), menos amostras de treinamento são extraídas pela janela deslizante, quando o `stride` acompanha o `max_length` (sem sobreposição). A relação é aproximadamente inversa: amostras $\approx (N - \text{ctx})/\text{ctx}$, onde N é o número total de tokens do corpus.

Os resultados obtidos sobre o corpus de teste ($N = 5\,145$ tokens BPE) confirmam essa relação:

Contexto (max_length)	4	32	128
Amostras geradas (<code>stride=ctx</code>)	1 286	160	40

Ou seja, multiplicar o contexto por 32 (de 4 para 128) dividiu o número de amostras por aproximadamente 32. Além disso, o parâmetro `stride` modula essa relação de forma independente: fixando o contexto em 32 e variando o `stride` de 32 (sem sobreposição) para 8 (75% de sobreposição entre janelas), o número de amostras sobe de 160 para 640 — quatro vezes mais, à custa de conteúdo redundante entre amostras vizinhas. Há, portanto, um compromisso entre **quantidade** de amostras e **diversidade** (independência estatística) entre elas.

Questão 8. Qual é o impacto da dimensão do embedding sobre as estruturas utilizadas pelo modelo?

A dimensão do embedding (`output_dim`) determina o tamanho do último eixo de todo tensor produzido a partir daí no pipeline: o *input embedding* tem forma (`batch_size`, `context_length`, `output_dim`). Aumentar `output_dim` não altera o número de tokens processados, mas aumenta:

- o número de parâmetros treináveis nas tabelas de embedding, de forma linear (`vocab_size × output_dim` para a tabela de tokens);
- o custo computacional (tempo) de cada consulta/soma, também de forma aproximadamente linear, como observado no experimento: de 16 para 768 dimensões, o tempo de montagem do *input embedding* para o mesmo lote cresceu de 0,138 ms para 1,388 ms — um fator de aproximadamente 10x, contra um fator de 48x no número de dimensões, o que sugere que, nessa escala pequena, outros custos fixos (indexação, alocação) ainda dominam o tempo total;
- a quantidade de memória necessária para armazenar cada representação intermediária.

Na prática, `output_dim` é a principal alavanca de capacidade representacional do modelo nesta etapa: dimensões maiores permitem capturar relações semânticas mais ricas entre tokens, ao custo de mais parâmetros e mais computação — o mesmo compromisso que, em escala muito maior, explica por que modelos GPT maiores (com `output_dim` de 768 a mais de 12 000) são simultaneamente mais capazes e mais custosos.

Questão 9. Qual é a função do DataLoader no pipeline?

O `DataLoader` do PyTorch é a camada de organização entre o *Dataset* (que sabe produzir um par entrada/alvo por índice) e o laço de treinamento (que precisa consumir lotes de exemplos). Suas responsabilidades específicas, observadas na implementação (`src/dataset.py`), são:

- **Agrupar** `batch_size` amostras individuais em um único tensor, de forma (`batch_size`, `context_length`) — por exemplo, `batch_size=8` produz lotes de forma (8, 4) para `max_length=4`;
- **Embaralhar** (`shuffle=True`) a ordem das amostras a cada época, evitando que o modelo aprenda uma ordem espúria de apresentação dos dados;
- **Descartar** o último lote incompleto de uma época (`drop_last=True`), garantindo que todos os lotes tenham exatamente o mesmo formato, o que simplifica e acelera o treinamento em GPU.

Sem o `DataLoader`, seria necessário implementar manualmente a lógica de particionamento, embaralhamento e formatação em lote — o que o torna, na prática, o componente que efetivamente conecta os dados

preparados ao restante da arquitetura.

Questão 10. Quais informações produzidas nesta Sprint serão utilizadas pelo mecanismo de atenção da Sprint seguinte?

O artefato final produzido por esta Sprint é o *input embedding*: um tensor de forma $(batch_size, context_length, output_dim)$, resultado da soma entre o token embedding e o positional embedding de cada posição. É exatamente esse tensor que o Capítulo 3 (*Coding Attention Mechanisms*) toma como ponto de partida.

O mecanismo de self-attention projeta cada vetor de entrada em três papéis — *query*, *key* e *value* — por meio de matrizes de peso treináveis aplicadas sobre esse mesmo *output_dim*. Ou seja, a dimensão de embedding escolhida nesta Sprint (por exemplo, 256) se torna diretamente a dimensão de entrada das matrizes de projeção da Sprint 3. Da mesma forma, o *context_length* usado aqui para montar as sequências de treinamento define o tamanho da matriz de pontuações de atenção ($context_length \times context_length$) e da máscara causal que impede cada posição de “ver” tokens futuros. Em resumo, a Sprint 2 entrega tanto o **conteúdo** (via token embedding) quanto o **contexto posicional** (via positional embedding) que o mecanismo de atenção precisará ponderar dinamicamente na Sprint 3.